

## CCAPDEV Term 2, AY 2023 – 2024

### Make-up Mini Challenge 3: Debt Tracker

#### General Instructions

- Read the instructions first before proceeding!
- You will be given a template project. Once downloaded, simply run the command ``npm install`` to install the project's dependencies.
- You may use the internet and your notes for reference, but you may not communicate with any other person for help.
- For any clarifications, ask your instructor.

#### Submission Details

Zip all your files and name it `MC3_LastnameFirstname.zip`. Submit the zipped folder to AnimoSpace.

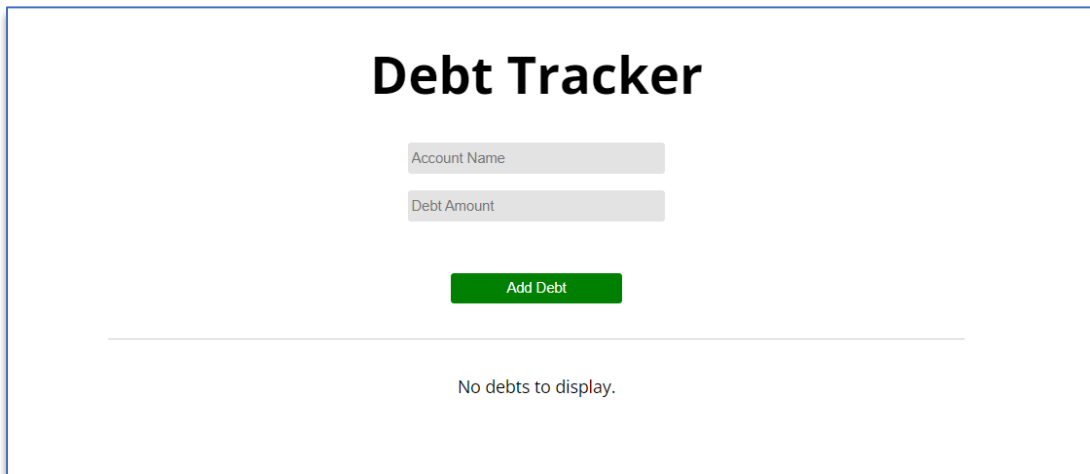
#### SPECIFICATIONS

The webpage you will be recreating is an Election Ballot Counter web app. For this mini challenge, you are already provided the starter files required. The layouts and styling are already completely defined, you only have to complete its functionality.

For the activity, you are **expected to modify** the following files:

- `public/js/debtForm.js`
- `src/models/Account.js`
- `src/routes/mainRouter.js`
- `srs/views/index.hbs`

However, if your approach calls for editing files outside of the aforementioned, then you may do so.

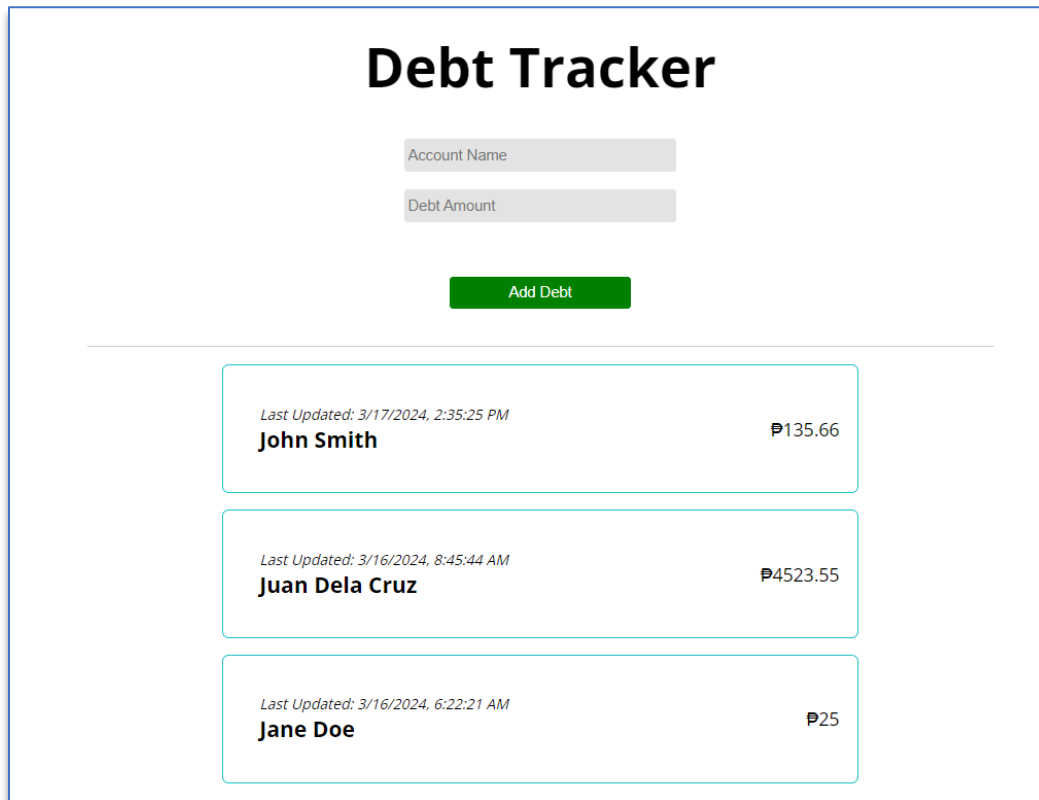


The screenshot shows a web application titled "Debt Tracker". It features two input fields: "Account Name" and "Debt Amount", both with light gray borders. Below these fields is a green button with the text "Add Debt". A horizontal line separates the input section from the output section, which displays the text "No debts to display." in a small, gray font.

**Figure 1. Debt Tracker Web Application**

## Feature 1: Retrieve Debt List

Upon accessing the root page, `http://localhost:3000/`, The user is presented a form and a list of accounts with outstanding debts (See Figure 2 below). Each account item contains the following information: (1) the account name, (2) the amount of debt, and (3) the date the debt has been last updated. The account items should be sorted by the last updated field in descending order (i.e., the most recent debt is at the top).



The screenshot shows a web application titled "Debt Tracker". At the top, there is a form with two input fields: "Account Name" and "Debt Amount", followed by a green "Add Debt" button. Below the form, there is a list of three accounts, each displayed in a light blue box. Each box contains the "Last Updated" date and time, the account name, and the debt amount in Philippine Pesos (₱).

| Last Updated          | Account Name   | Debt Amount |
|-----------------------|----------------|-------------|
| 3/17/2024, 2:35:25 PM | John Smith     | ₱135.66     |
| 3/16/2024, 8:45:44 AM | Juan Dela Cruz | ₱4523.55    |
| 3/16/2024, 6:22:21 AM | Jane Doe       | ₱25         |

**Figure 2. Homepage with Populated Accounts with Outstanding Debt**

If the account list in the database is empty, however, the page should instead display the paragraph 'No debts to display.' (as seen in Figure 1).

Note: A script has been prepared to populate the database with sample data, which is `./populateDb.js`. Do note that the `accountSchema` from `./src/models/Account.js` must be setup first before this script can work. You can execute this script by simply running the command `"npm run populate"`.

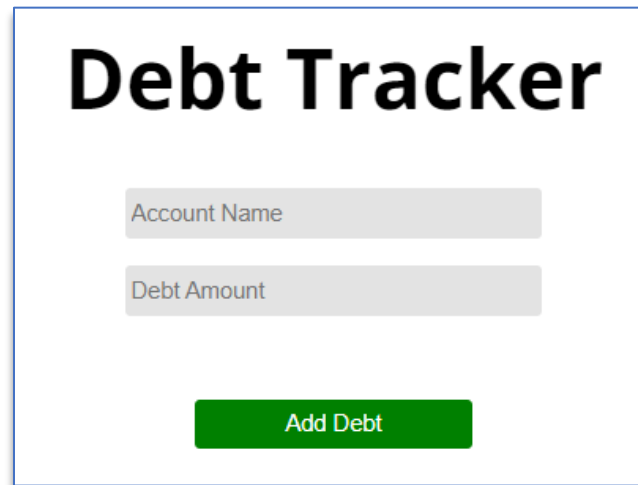
## Feature 2: Add Debt

The user is provided a form which they can use to add new **debts** into the system. The form asks the user to enter the **account name** for the debt, and the **debt amount**.

Upon submitting, if the given account name does not exist yet in the database, a new account is created with the given debt amount. Otherwise, if the account already exists,

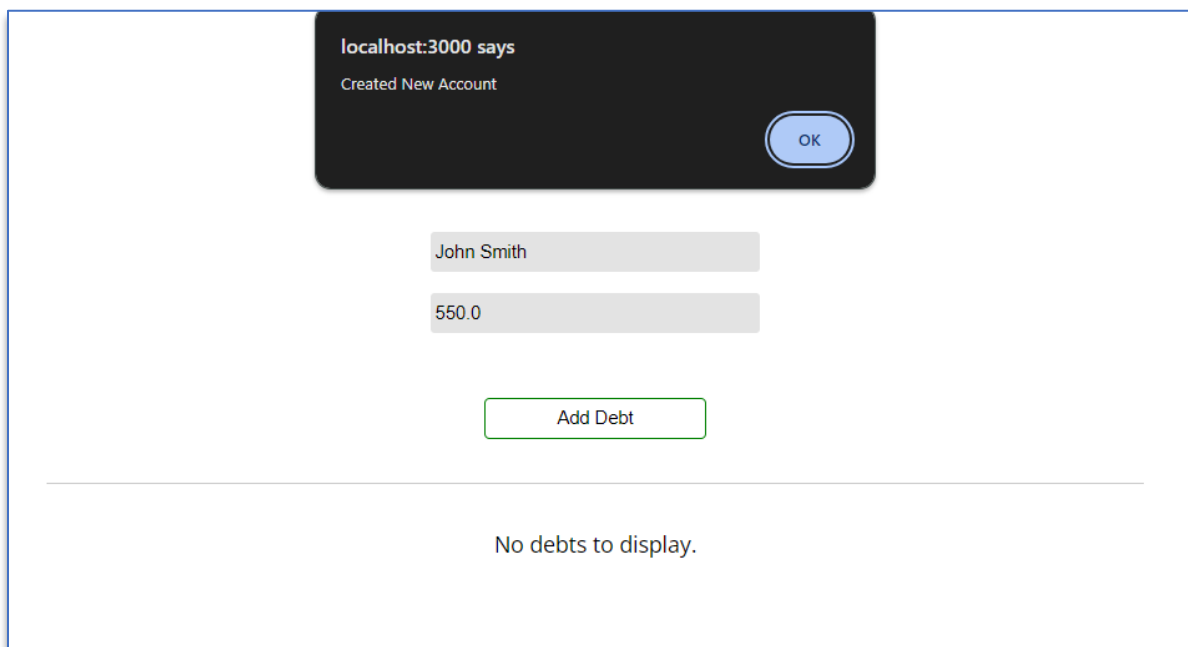
the debt amount is simply added to that account's existing debt. For both instances, the lastUpdated date field should be updated and set to the current datetime.

There is no limit to how many debts can be added. After a debt is **successfully** added, the web app displays an alert message which indicates whether an account has been newly created or updated, then the page is refreshed (see Figure 4 and 5).



A web form titled "Debt Tracker" with a large black heading. Below the heading are two input fields: "Account Name" and "Debt Amount", both with light gray borders. Below these fields is a green button with the text "Add Debt".

**Figure 3. User enters a new serial number in the form**



The same "Debt Tracker" form as in Figure 3, but with an alert message displayed at the top. The alert is a dark gray box with the text "localhost:3000 says" and "Created New Account" in white. There is a blue "OK" button in the bottom right corner of the alert. Below the alert, the "Account Name" field now contains the text "John Smith" and the "Debt Amount" field contains the text "550.0". The "Add Debt" button is now outlined in green. Below the form fields, there is a horizontal line and the text "No debts to display."

**Figure 4. Created new account after adding debt**

The screenshot displays a web application interface. At the top, a dark grey toast notification box contains the text "localhost:3000 says" and "Updated Existing Account", with an "OK" button on the right. Below this, there are two input fields: the first contains "John Smith" and the second contains "100.23". A green-outlined button labeled "Add Debt" is positioned below the inputs. A horizontal line separates this section from the bottom section. The bottom section features a light blue box containing the text "Last Updated: 3/19/2024, 9:10:59 PM" and "John Smith" on the left, and "₱550" on the right.

**Figure 5. Updated an existing account after adding debt**

### **Feature 3: Form Validation**

Before the debt can be added, the form must validate whether the entered inputs are correct. If the inputs are invalid, an error message (see `index.hbs`) is displayed on the page (see Figure 6). The following are the corresponding error messages for given error scenarios:

| Error Scenario                      | Error Message                          |
|-------------------------------------|----------------------------------------|
| accountName is empty                | Input is missing value(s)!             |
| debtAmount is empty                 |                                        |
| debtAmount is not a positive number | Debt amount must be a positive number! |

The screenshot shows a web application titled "Debt Tracker". It features two input fields: the first contains "Jane Doe" and the second is labeled "Debt Amount" and is currently empty. Below the inputs, a red error message "Input is missing value(s)!" is displayed. A green button labeled "Add Debt" is located at the bottom of the form.

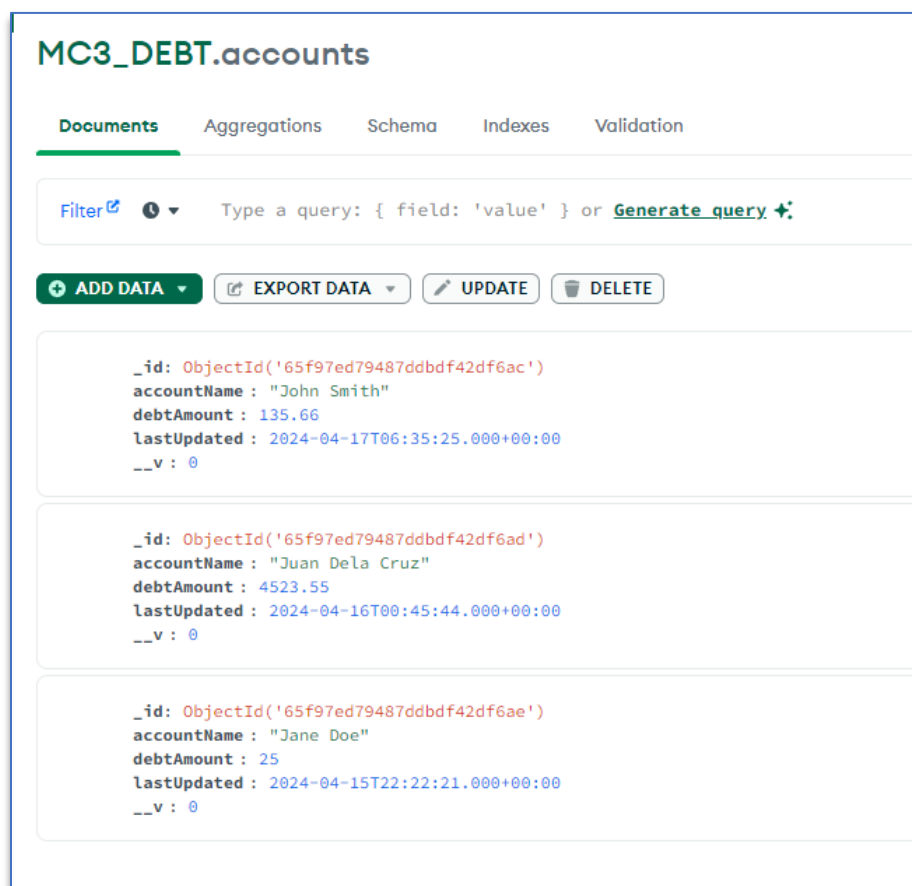
**Figure 6. Error Message for missing value**

## Document Structure

Finally, the documents from the accounts collection must adhere to the Account schema to be defined in `./src/models/Account.js`. The account schema should have the following fields:

- `accountName` – String. Represents the name of the account owner. Should be required and unique.
- `debtAmount` – Number. Represents the amount of outstanding debt the account has. Should be required.
- `lastUpdated` – Date. Represents the last time the debt on this account has been updated (or created). No additional constraints aside from the type.

See Figure 7 below for examples on how the documents look like as stored in MongoDB.



**Figure 7. Accounts, as stored in the database**

---

For this activity, it is your decision whether to use jQuery or not. The activity is accomplishable with either jQuery or vanilla JavaScript.

The starter code uses CommonJS modules syntax, if you wish to use the ES6 syntax, you may update the package.json and the existing JavaScript files.

The server is hosted on port 3000, and the data is stored on the MC3\_DEBT database, as defined from the .env file.

---

---

### **RUBRIC FOR GRADING**

| <b>Content</b>                                                                                                                          | <b>Score</b> |
|-----------------------------------------------------------------------------------------------------------------------------------------|--------------|
| <b>Retrieve Debt List</b>                                                                                                               | <b>15</b>    |
| → The account items are rendered properly (i.e., lastUpdated, accountName, and debtAmount are displayed correctly)                      | 5            |
| → The paragraph 'No debts to display' is rendered if the list is empty.                                                                 | 5            |
| → The account items are sorted by lastUpdated field in descending order                                                                 | 5            |
| <b>Add Debt</b>                                                                                                                         | <b>20</b>    |
| → The form data is sent and can be read from the server                                                                                 | 3            |
| → If the account name is new, the server reads the request and saves the created document                                               | 5            |
| → If the account name already exists, the server reads the request and updates the matching account document.                           | 5            |
| → On success, the server informs the client whether an account has been created <b>or</b> an existing account is updated.               | 3            |
| → On success, an alert message is displayed, indicating whether an account has been created or updated <b>before</b> reloading the page | 4            |
| <b>Form Validation</b>                                                                                                                  | <b>5</b>     |
| → Error message for empty accountName and/or debtAmount                                                                                 | 3            |
| → Error message for non-positive debtAmount                                                                                             | 2            |
| <b>Total</b>                                                                                                                            | <b>40</b>    |